CS208 Report - Complexity in Action

Finlay Robb • 202400218 • CES
CS208 Individual Coursework Assignment 1

This report will cover benchmarking selection sort and merge sort algorithms using instruction counting.

1. Introduction

When writing software, it is important to understand how the program will perform, and how changes to the code can speed up or slow down execution.

This is done through benchmarking the program, with the most common method being measuring how long the program takes to run from start to finish, called wall time. However, this method is fundamentally limited by the fact that this is not a very precise or repeatable test, since the results depend entirely on the computer it was run on and what else that computer might have been doing at the time. This means that only results run on the same system can be directly compared, and even then the average of many runs should be used to get a more accurate result. This means that small performance changes might not be noticed, and the output is typically quite noisy, with a high variance between runs.

An alternative to wall time is instruction counting. At a basic level, this simply counts every assembly instruction executed during the program, returning the total. This means that the profiling output should be exactly the same every time it is run, assuming that the input and program are unchanged between runs. This also has the benefit that the results should be hardware agnostic, so the results should be perfectly replicable. For these reasons, instruction counting will be used as the primary profiling method in this report. However, it is important to acknowledge that instruction counting may not always line up with the actual time a program takes to run, due to factors such as different instructions taking longer than others, and memory access time not being considered.

In this report, 3 separate kinds of arrays will be tested: random, linear, and reversed. Random indicates that each element in the array should be a random number from –1000 to 1000 inclusive. Linear indicates a “best case” presorted array, and reversed indicates a “worst case” reverse sorted array.

2. Prerequisites

To perform the instruction counting, the Callgrind profiler, part of the Valgrind program, was used. This is a Linux program that records the call history of functions during a program run as a call-graph, and uses that to calculate the instruction count of a program. To interface with Callgrind, Gungraun was used, which is a rust library that connects the built-in rust benchmarking to Callgrind, to record the instruction count of a Rust program. Rust was chosen since there is zero overhead from garbage collection, and it is a relatively low-level language, so code corresponds very closely to the hardware, so the results should be very accurate.

To generate a graph of the data, the Matplotlib python library was used. This was chosen as it is a very mature and fully featured library, and python is very suited to this sort of task.

3. Procedure

First, an implementation of selection sort and merge sort were written.

Next the benchmarking framework was built, which operates as follows:

- First, the type of benchmark to run is specified - random, linear, or reversed.
- A list of lengths of arrays to test is read in, in this report lengths from 0 to 100 inclusive will be tested.
- During the benchmarking, for each sorting algorithm, and for each length of array to sort, a new array of the specified type and length is generated.
- This generated array is saved to a file, which could later be used to verify the results or debug problems.
- The instruction counter is started, and the sorting algorithm sorts the array.
- Once it is finished, the instruction counter is stopped, the array is checked to verify all elements are in order, and the sorted data is also saved to a file.
- The logfile generated by Callgrind containing the results is parsed to get the instruction count for that run, which is saved to a file.
- The benchmark is repeated a certain number of times to increase the accuracy of the results.
- After all the benchmarks have been run, a python script generates a graph containing all the data from the runs.
- This graph contains all the runs of that benchmark type, shown as a line graph.The max delta between the results is also shown on the graph as a shaded area around the line.

The crossover point, where the algorithms cross, is automatically calculated and added to the graph.

Finally, this graph is shown to the user and saved to a file.

4. Results

The results of the benchmarks are shown in the graphs below, with an arrow pointing to the crossover point, and the max delta shown as a shaded area around the lines.

For the random array, shown in Figure 1, the crossover point occurs at 66 elements, with selection sort being faster for arrays below this length, and merge sort being faster for arrays above this length.

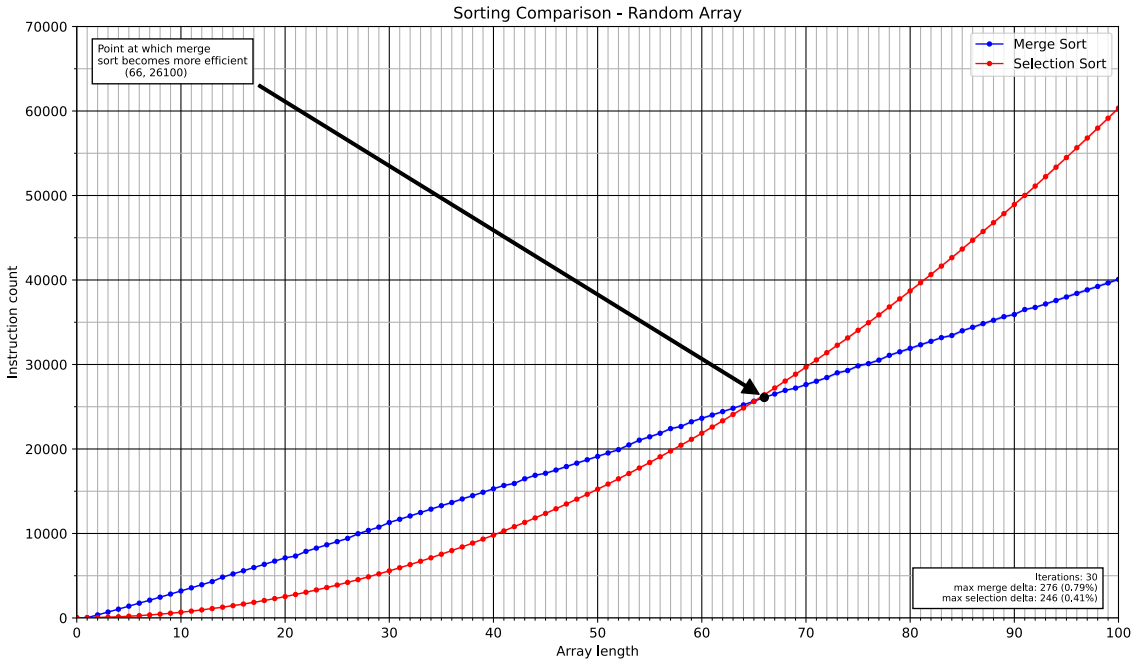

Figure 1: Merge vs Selection sort for a randomly sorted array

Even though the delta between the largest and smallest value for each run of the benchmark is shown on the graph in Figure 1, it is so small that it is hard to see. Therefore, a zoomed in version of the graph is shown in Figure 2, which shows the delta more clearly. Even though a different random array was used in every run, the results are still very consistent, with the maximum delta being only 0.79% for merge sort, and 0.41% for selection sort.

The benchmark was repeated 30 times for each length of array, this was due to time constraints (running with instruction counting massively increases the time it takes to run each benchmark), and the fact that instruction counting produces very consistent results.

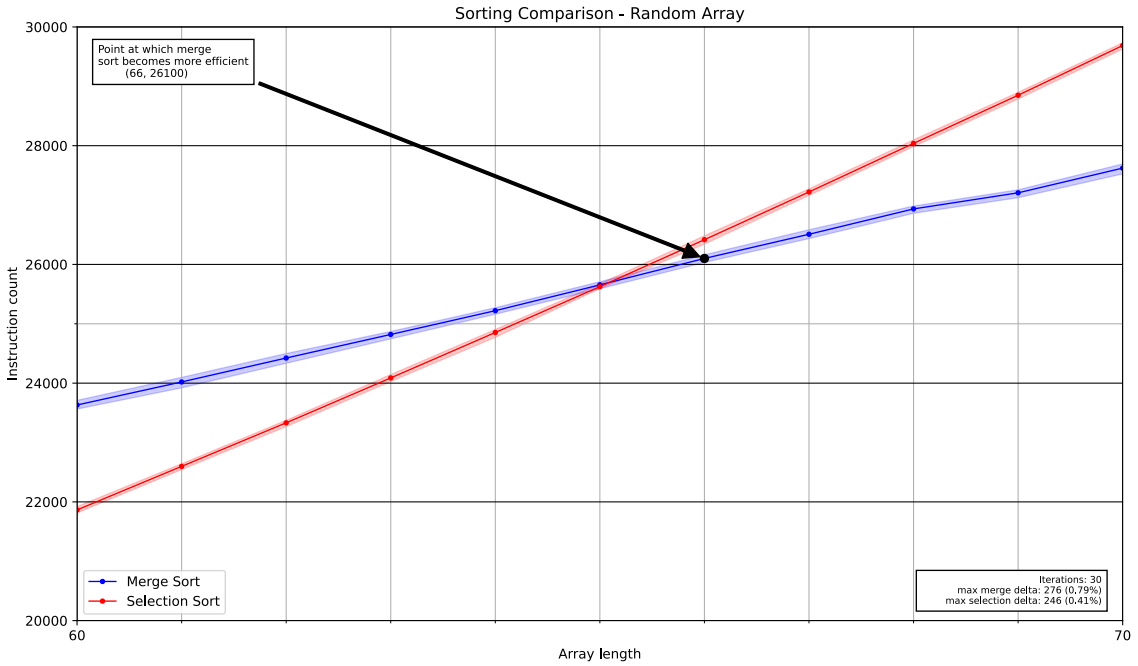

Figure 2: A zoomed in version of the random graph, showing the delta between runs more clearly

Below, Figure 3 and Figure 4 show the results for the linear and reversed arrays respectively. The crossover point stays roughly similar, being at 62 elements for the linear array, and 68 elements for the reversed array.

The delta between runs in these graphs is zero, which shows the benefit of instruction counting - the results are exactly the same for each run, provided the input is the same every time. This is also why these benchmarks were only repeated 10 times.

One interesting observation is that the instruction count for selection sort is significantly lower when sorting a reversed array, compared to a random or linear array, which are almost identical. This is because it will perform a swap at every iteration of the loop when sorting a reversed array, resulting in fewer instructions overall.

The results of the merge sort are more consistent, with random arrays being slightly higher than reversed and linear arrays, which are in turn slightly higher than linear arrays.

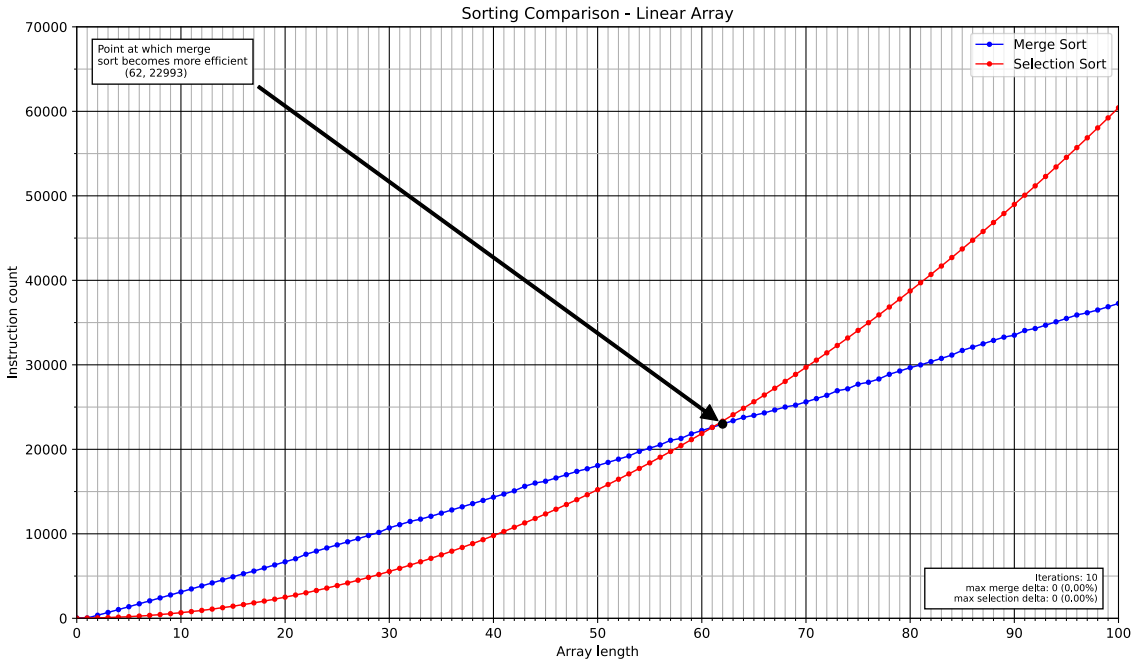

Figure 3: Merge vs Selection sort for a linear array

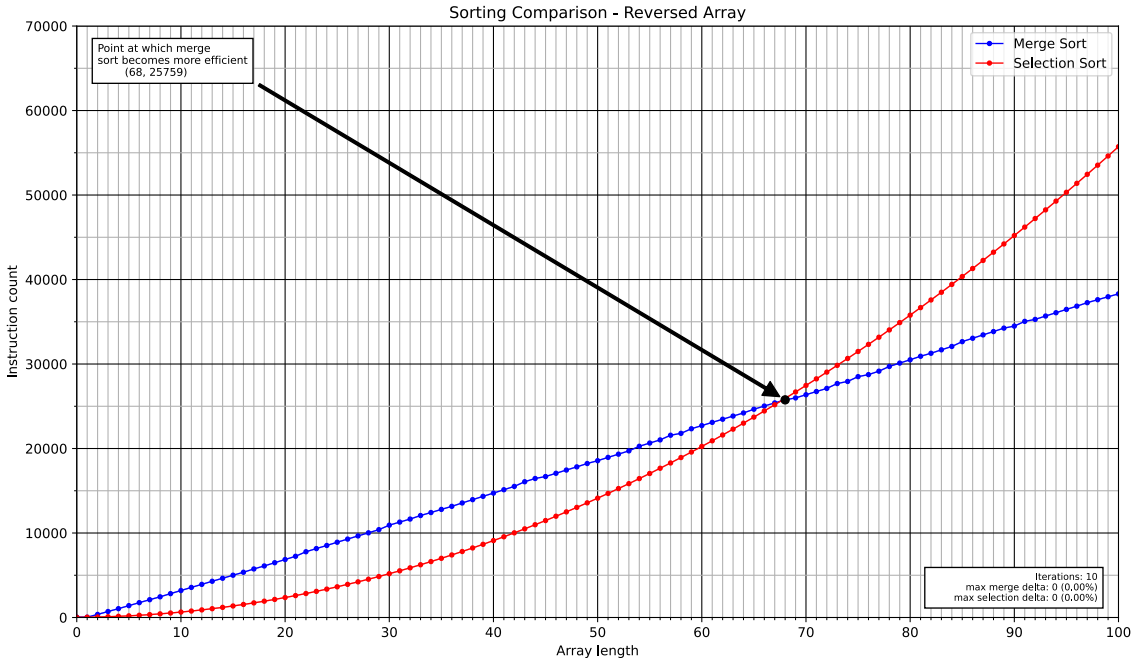

Figure 4: Merge vs Selection sort for a reversed array